Variations of the Turing Machine

(Deterministic) Turing Machine

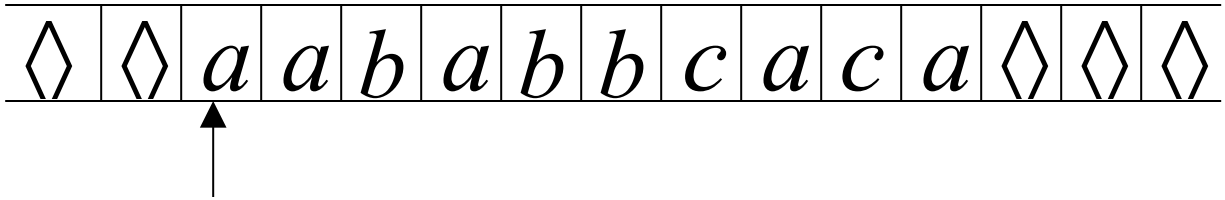
Formal Definition

A TM is $M = (Q, \Sigma, \Gamma, \vdash, \sqcup, \delta, s, t, r)$ where

- Q is a finite set of *states*,
- Σ is a finite *input alphabet*, used to encode the input string,
- Γ is a finite *tape alphabet* consisting of symbols written and read from the tape; $\Sigma \subsetneq \Gamma$,
- $\vdash \in \Gamma \setminus \Sigma$ is the *left endmarker*,
- $\sqcup \in \Gamma \setminus \Sigma$ is the *blank symbol*,
- $s \in Q$ is the *start state*,
- $t \in Q$ is the unique *accepting state*,
- $r \in Q$ $(r \neq t)$ is the unique *rejecting state*,
- $\delta : (Q \setminus \{t, r\}) \times \Gamma \to Q \times \Gamma \times \{\mathrm{L}, \mathrm{R}\}$ is the transition function that never overwrites $\vdash$.

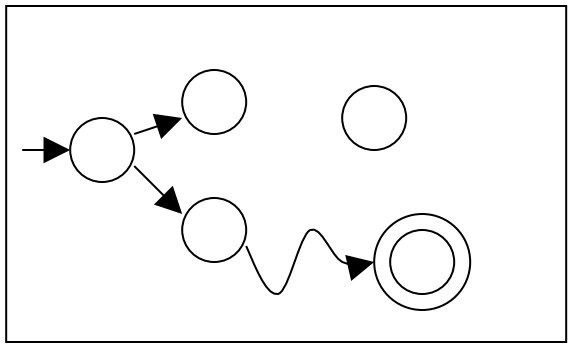
The Standard Model

Infinite Tape



Read-Write Head (Left or Right)

Control Unit



Deterministic

Variations of the Standard Model

Turing machines with:

- Stay-Option
- Semi-Infinite Tape
- Multitape
- Multidimensional
- Nondeterministic

Different Turing Machine **Classes**

Same Power of two machine classes:

both classes accept the
same set of languages

We will prove:

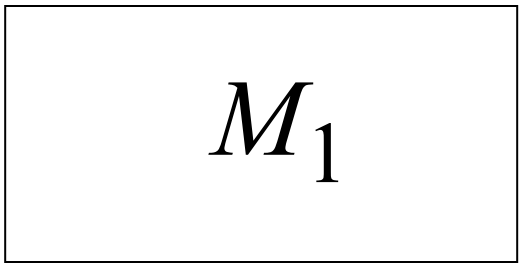
each new class has the same power
with Standard Turing Machine

(accept Turing-Recognizable Languages)

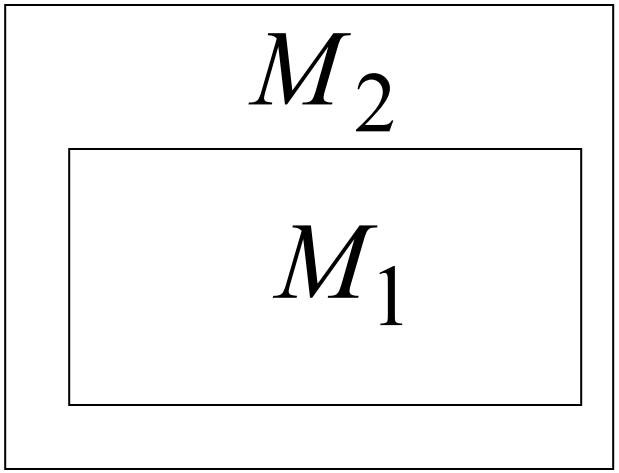
Simulation: A technique to prove same power.

Simulate the machine of one class with a machine of the other class

First Class
Original Machine



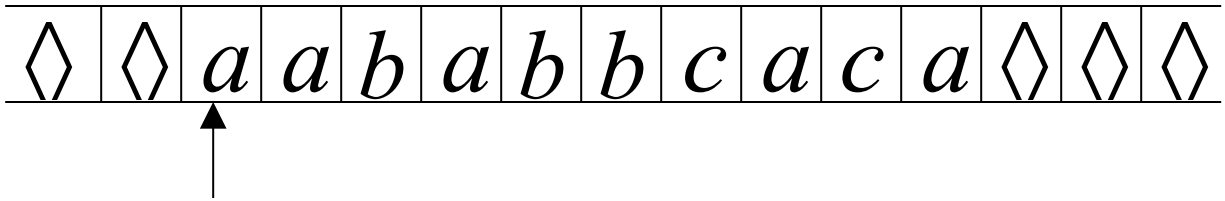
Second Class
Simulation Machine



simulates M_1

Turing Machines with Stay-Option

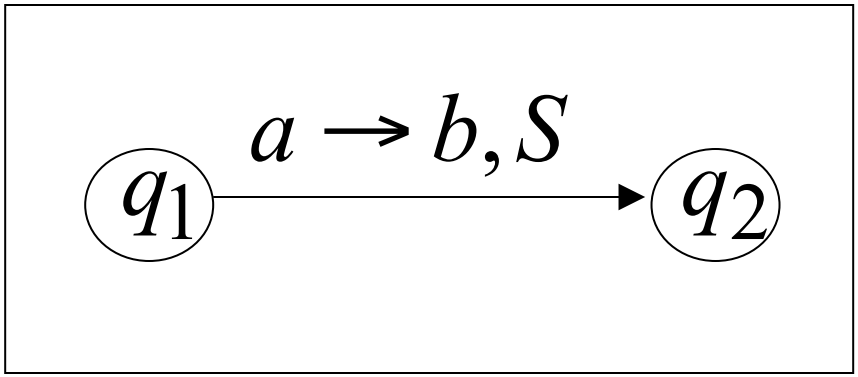
The head can stay in the same position



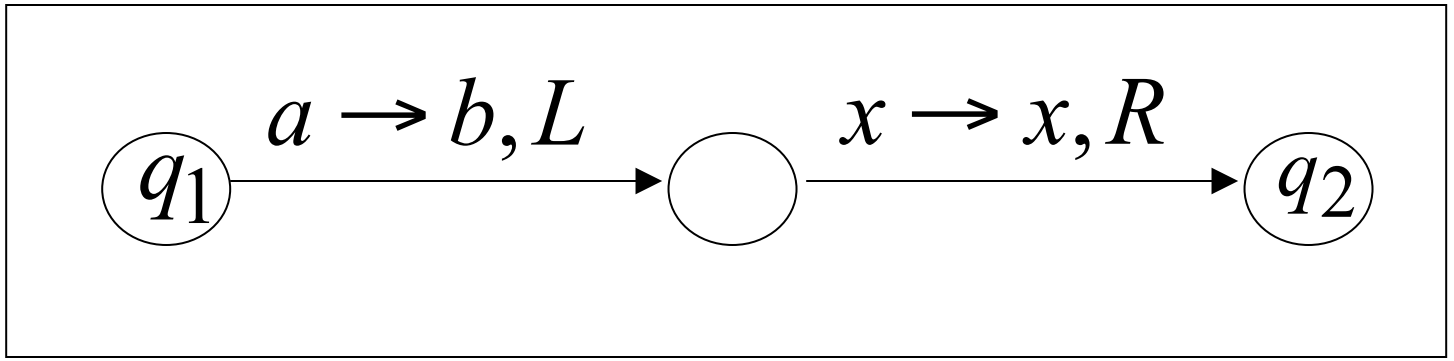
Left, Right, Stay

L,R,S: possible head moves

Stay-Option Machine



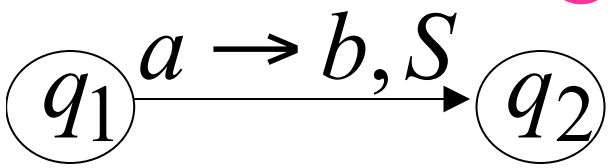
Simulation in Standard Machine

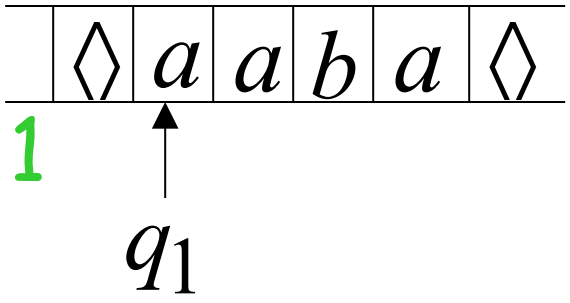


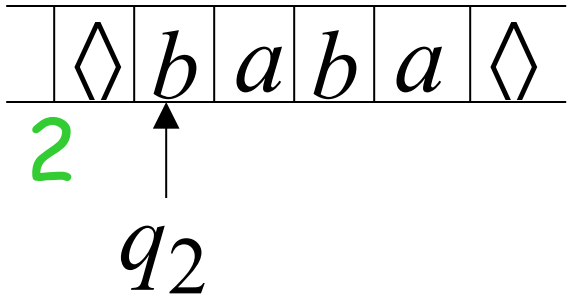
For every possible tape symbol x

example of simulation

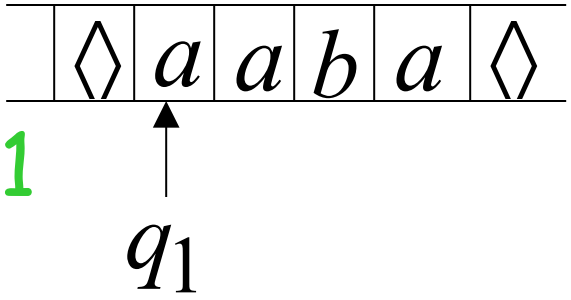
Stay-Option Machine:

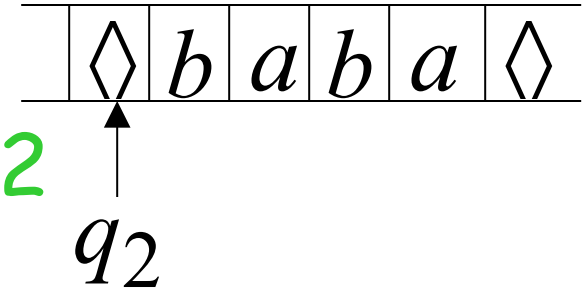


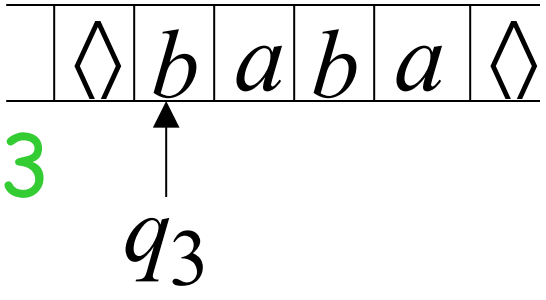




Simulation in Standard Machine:







END OF PROOF

Encoding the Tape Alphabet

It is possible to encode any finite alphabet into binary using a ***complete binary prefix code***.

Example. The alphabet $\{A, B, C\}$.

Use the following encoding:

$$A \mapsto 00, \qquad B \mapsto 01, \qquad C \mapsto 1.$$

So

$$A \;\; B \;\; A \;\; C \;\; B \;\; C \;\; \cdots$$

becomes

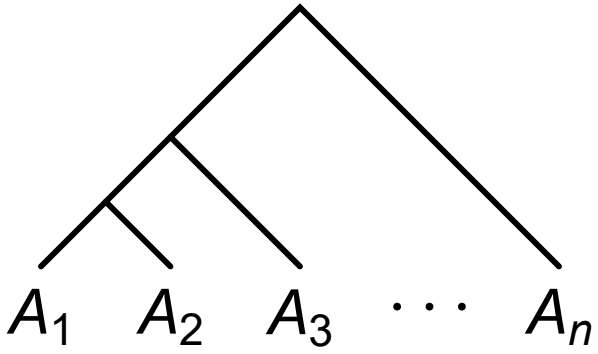
$$0 \;\; 0 \;\; 0 \;\; 1 \;\; 0 \;\; 0 \;\; 1 \;\; 0 \;\; 1 \;\; 1 \;\; \cdots$$

Encoding the Tape Alphabet

It is possible to encode any finite alphabet into binary using a ***complete binary prefix code***.

General Procedure. For the alphabet $\{A_1, A_2, \ldots, A_n\}$.

Choose a binary tree with one leaf for each letter:

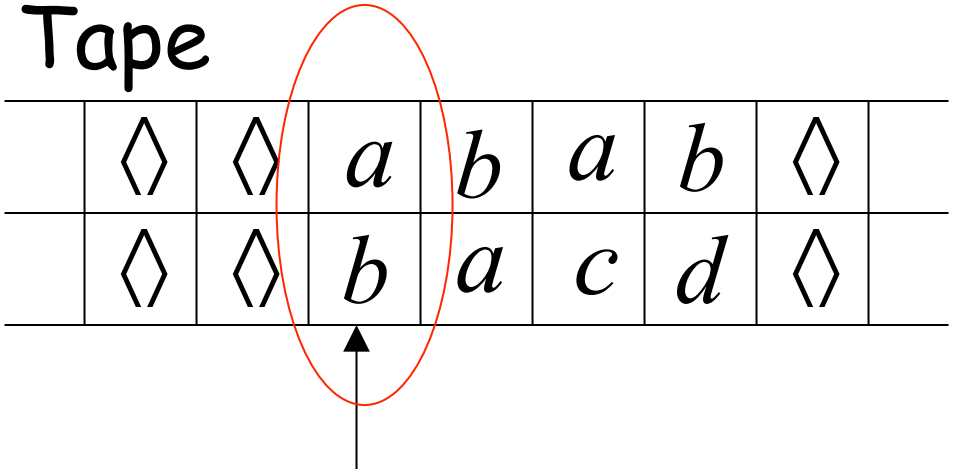


The binary code for each letter is given by its position in the tree.

A useful trick: Multiple Track Tape

helps for more complicated simulations

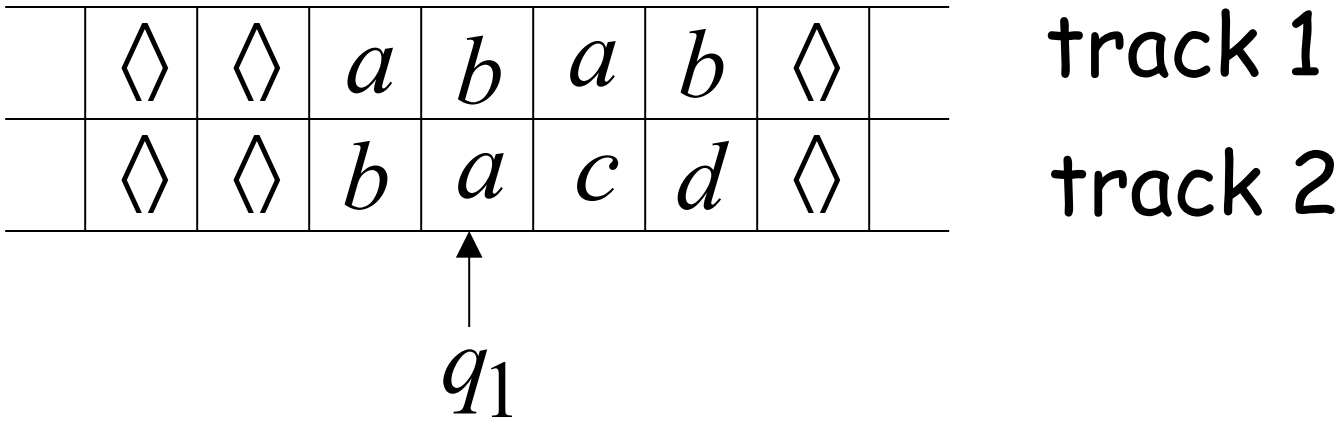
One Tape

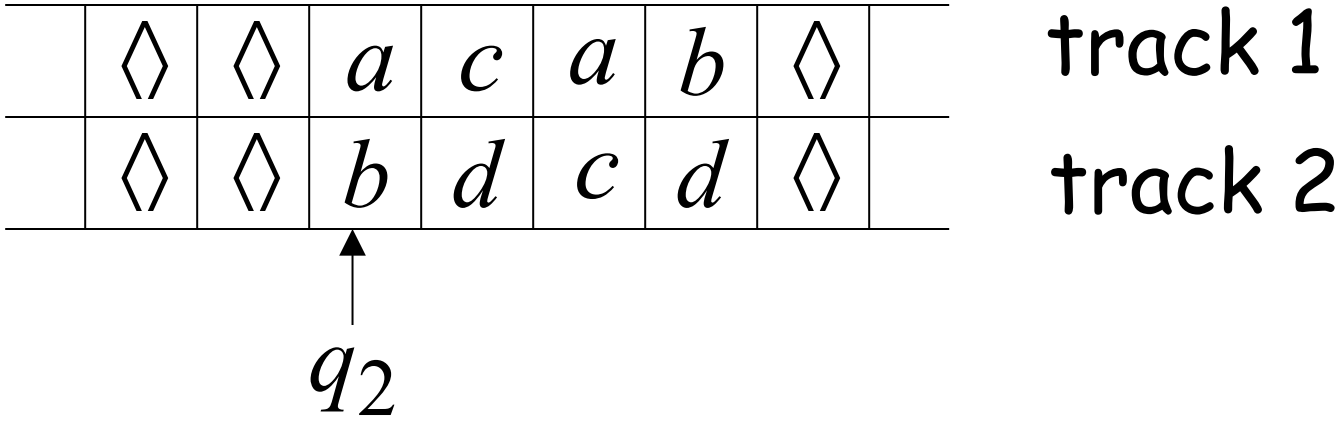


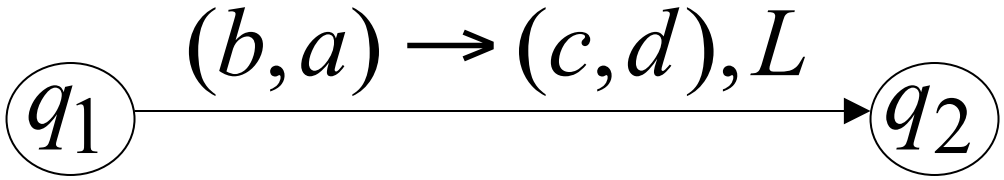
One head

One symbol (a,b)

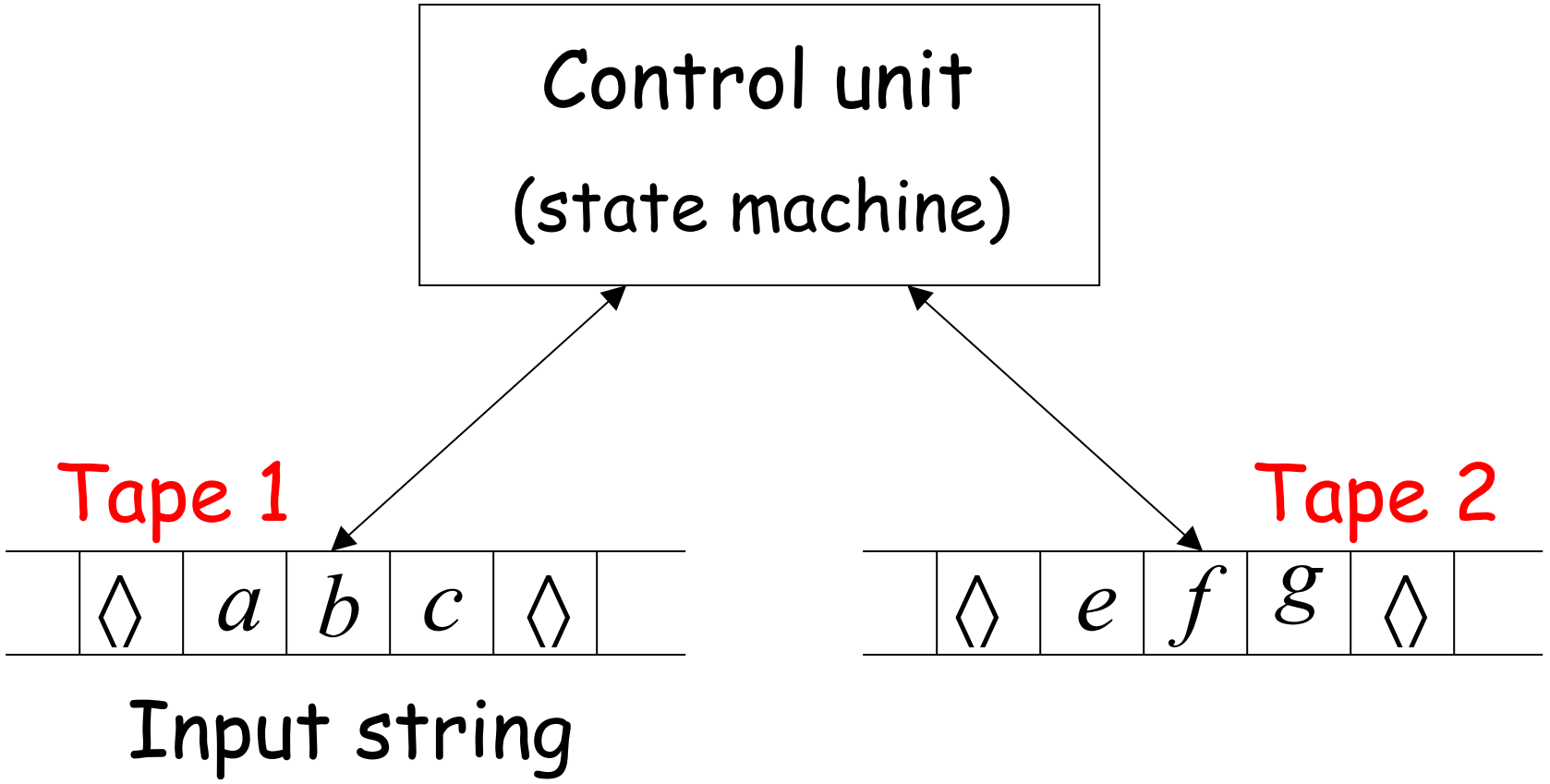
It is a standard Turing machine, but each tape alphabet symbol describes a pair of actual useful symbols


track 1
track 2
$\Diamond$ $\Diamond$ a b a b $\Diamond$
$\Diamond$ $\Diamond$ b a c d $\Diamond$
q_1

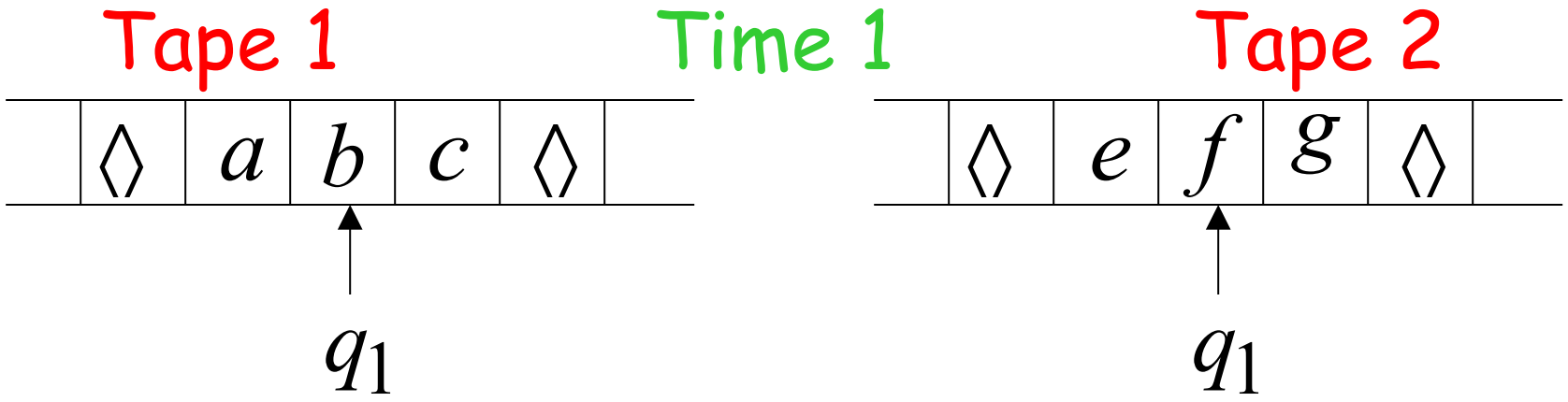

track 1
track 2
$\Diamond$ $\Diamond$ a c a b $\Diamond$
$\Diamond$ $\Diamond$ b d c d $\Diamond$
q_2

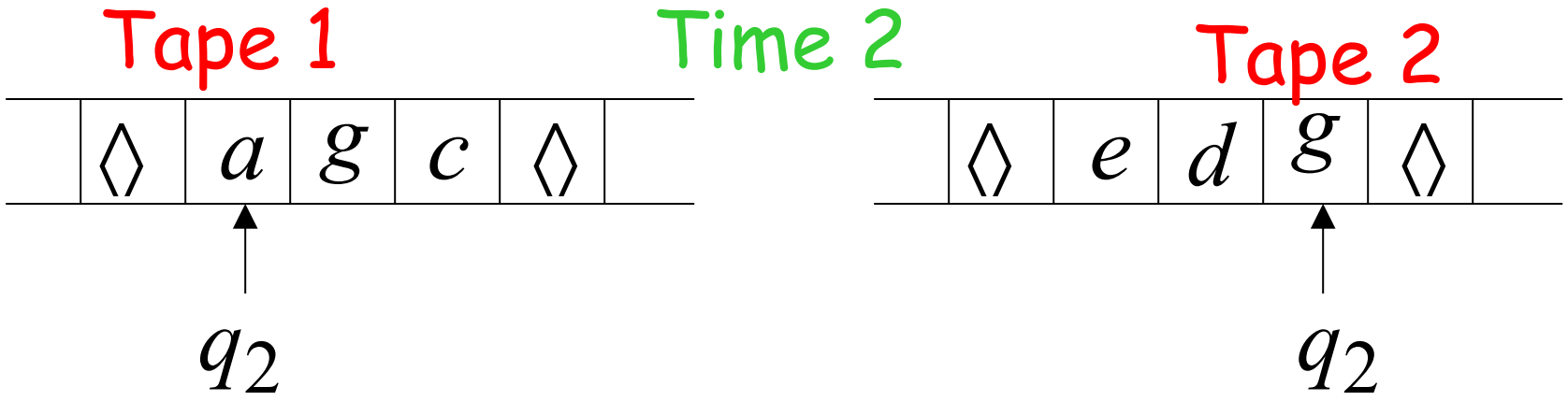

$(b,a) \rightarrow (c,d), L$
q_1
q_2

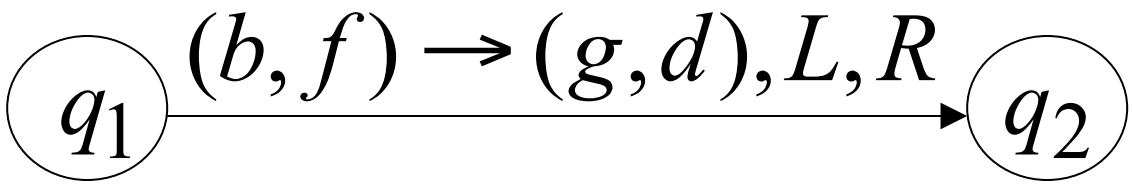
Multi-tape Turing Machines



Input string appears on Tape 1


Tape 1
Time 1
Tape 2
◊ a b c ◊
◊ e f g ◊
q_1
q_1


Tape 1
Time 2
Tape 2
◊ a g c ◊
◊ e d g ◊
q_2
q_2


q_1
$(b,f) \rightarrow (g,d), L, R$
q_2

Theorem: Multi-tape machines have the same power with Standard Turing machines

Proof: 1. Multi-tape machines simulate Standard Turing machines

2. Standard Turing machines simulate Multi-tape machines

1. Multi-tape machines simulate Standard Turing Machines:

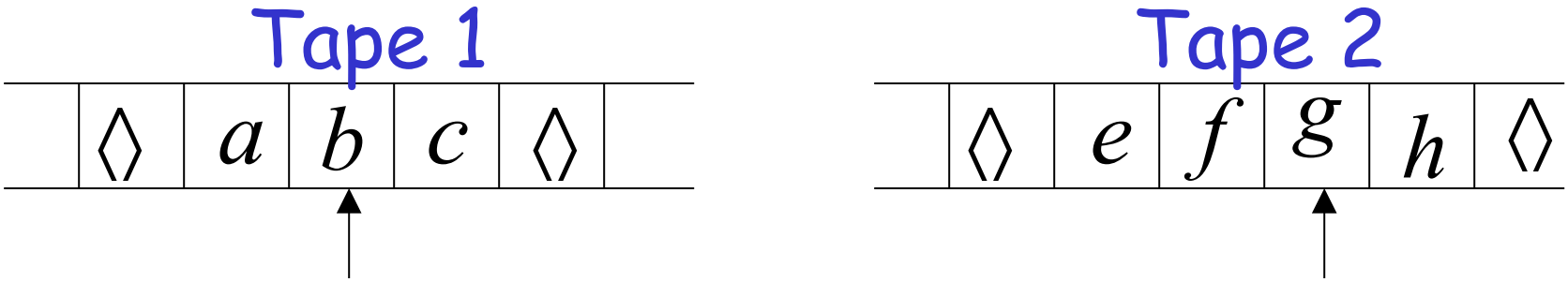
Trivial: Use one tape

2. Standard Turing machines simulate Multi-tape machines:

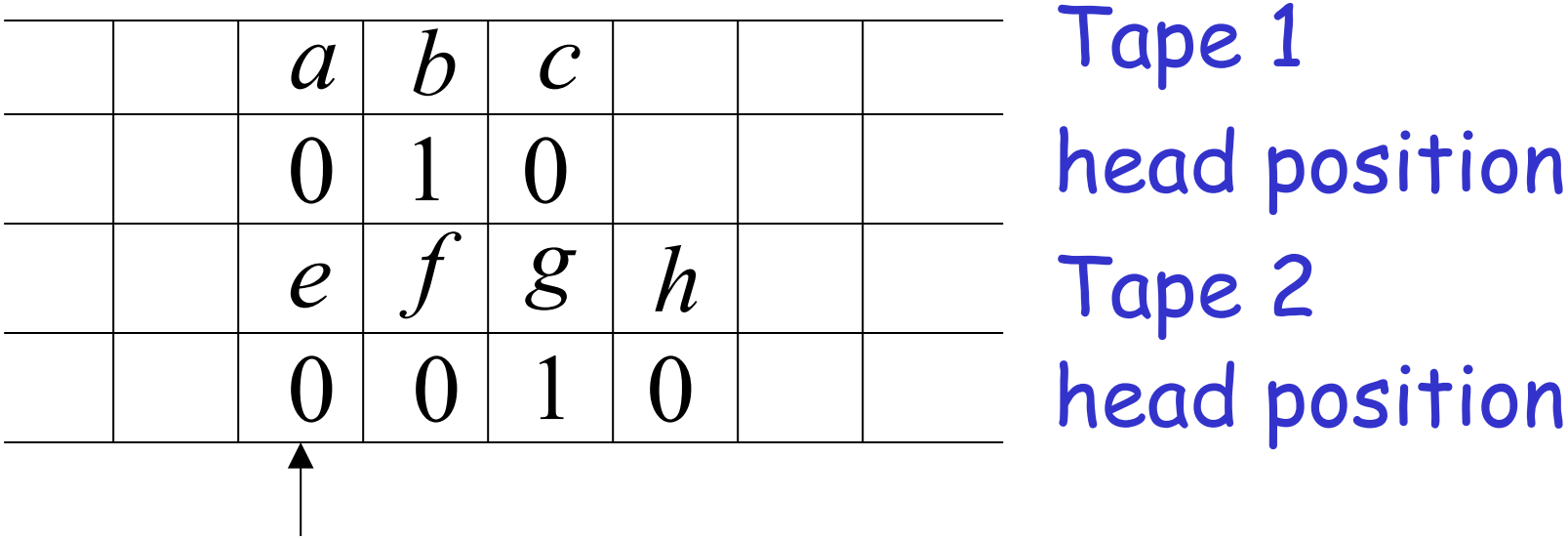
Standard machine:

- Uses a multi-track tape to simulate the multiple tapes
- A tape of the Multi-tape machine corresponds to a pair of tracks

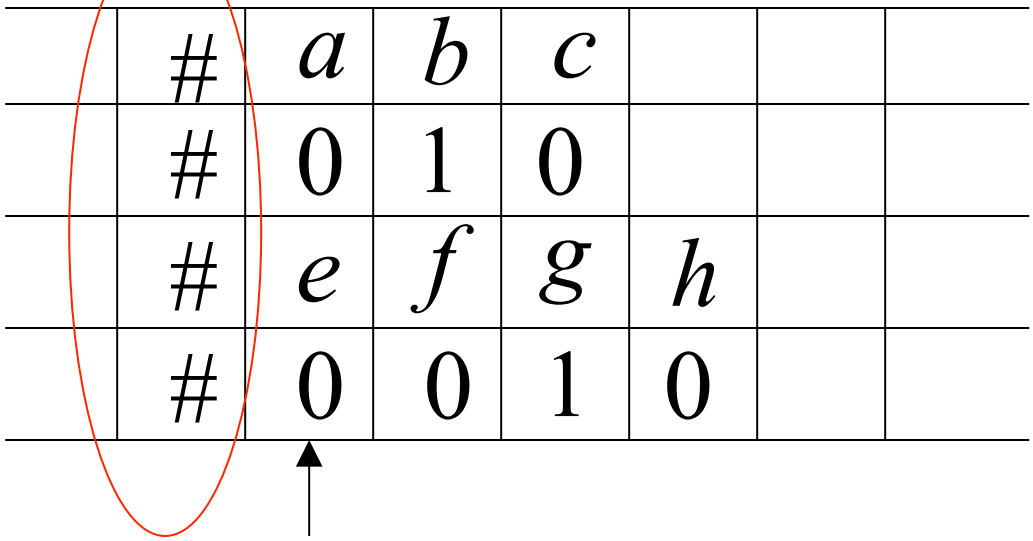
Multi-tape Machine



Standard machine with four track tape



Reference point



	#	a	b	c			
	#	0	1	0			
	#	e	f	g	h		
	#	0	0	1	0		

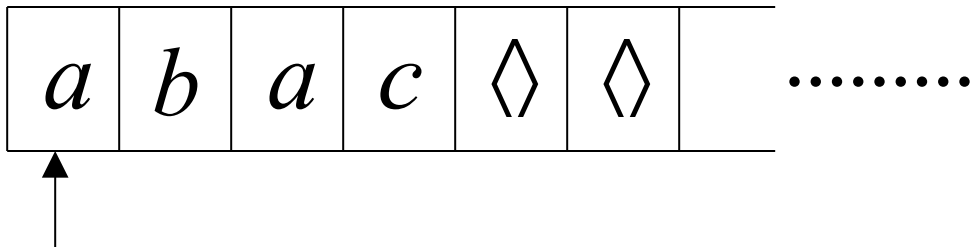
Repeat for each Multi-tape state transition:

1. Return to reference point
2. Find current symbol in Track 1 and update
3. Return to reference point
4. Find current symbol in Tape 2 and update

END OF PROOF

Semi-Infinite Tape

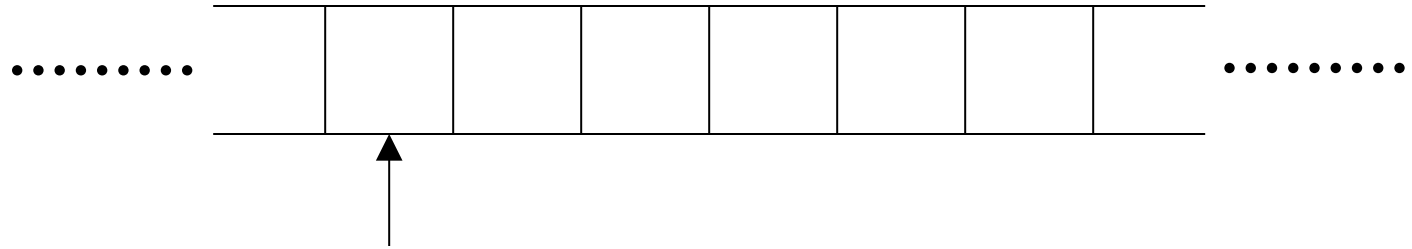
The head extends infinitely only to the right



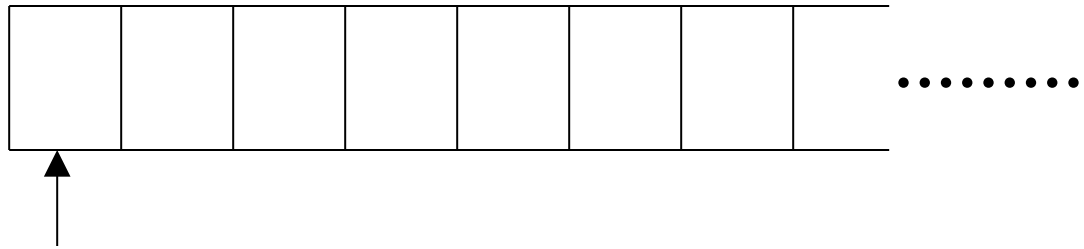
- Initial position is the leftmost cell
- When the head moves left from the border, it returns back to leftmost position

2. Semi-Infinite tape machines simulate Standard Turing machines:

Standard machine

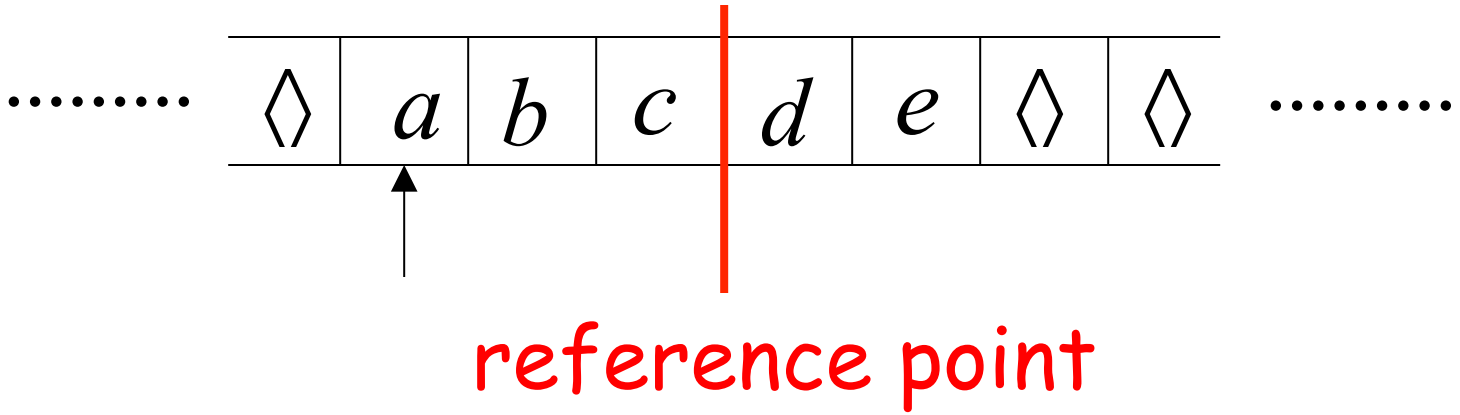


Semi-Infinite tape machine

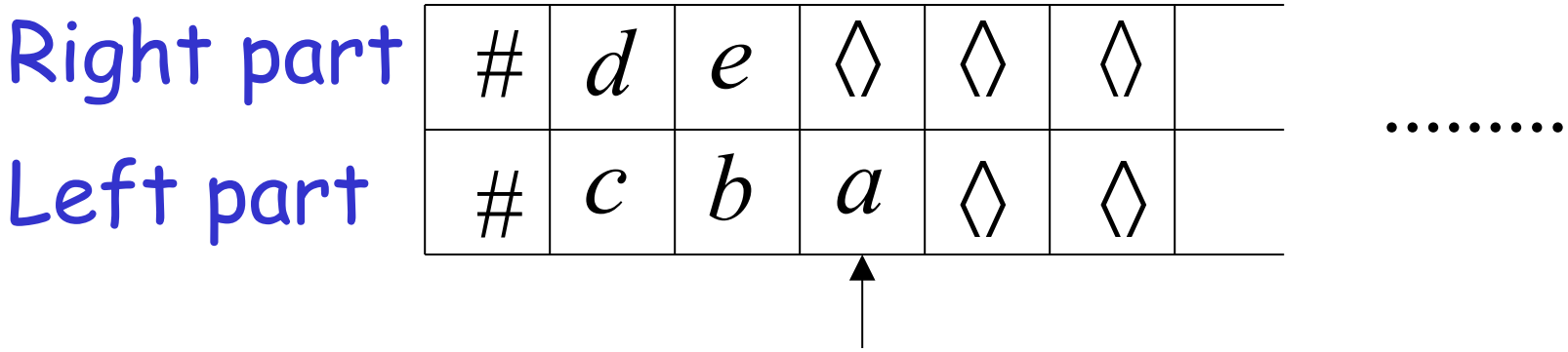


Squeeze infinity of both directions to one direction

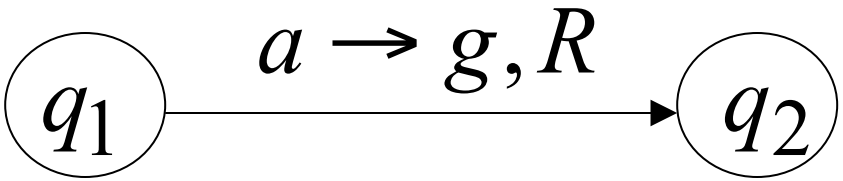
Standard machine



Semi-Infinite tape machine with two tracks

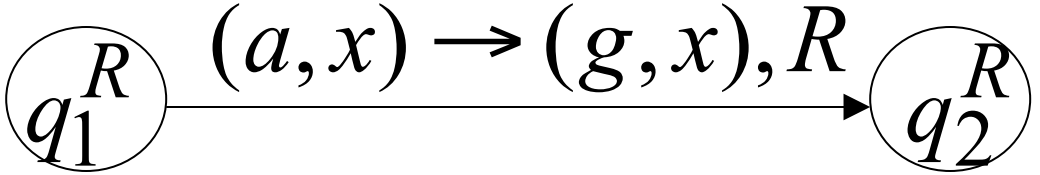


Standard machine

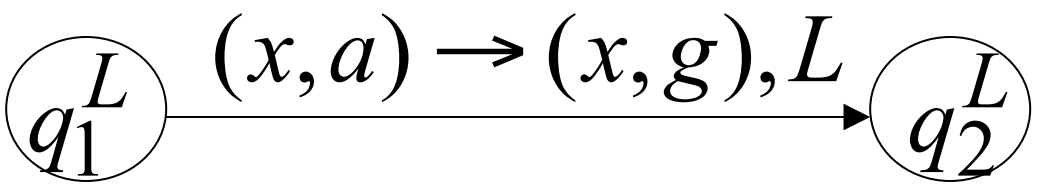


Semi-Infinite tape machine

Right part



Left part

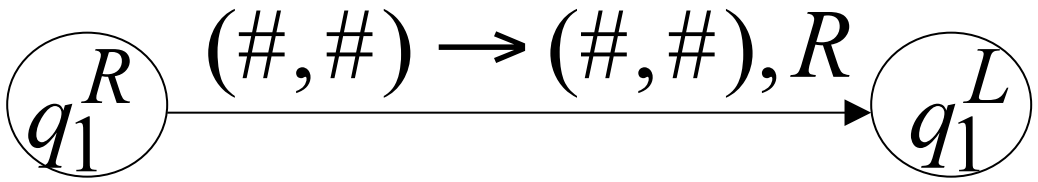


For all tape symbols X

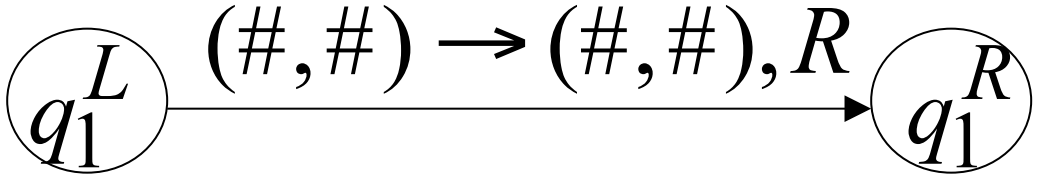
At the border:

Semi-Infinite tape machine

Right part

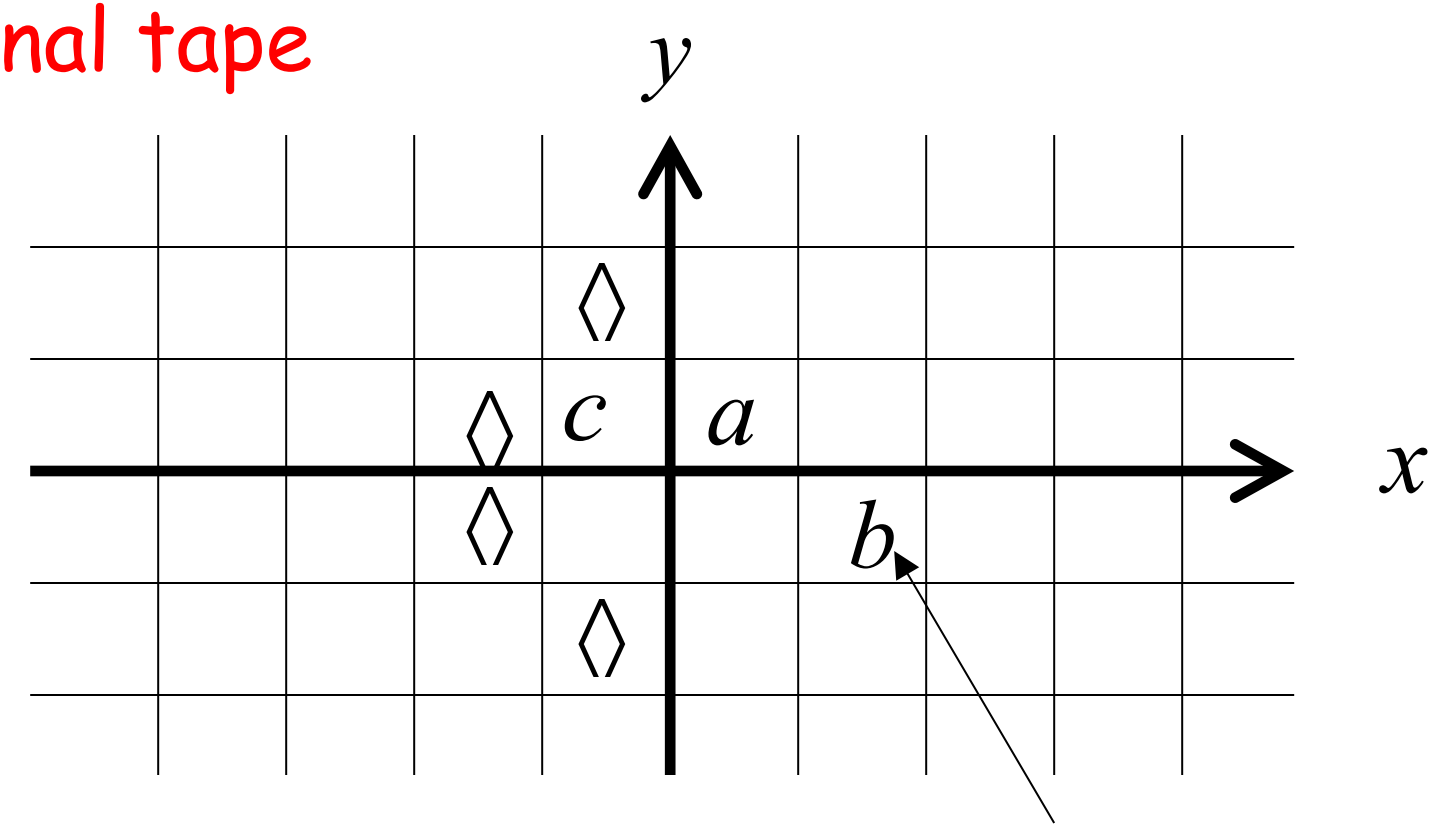


Left part



Multidimensional Turing Machines

2-dimensional tape



MOVES: L,R,U,D

U: up D: down

HEAD

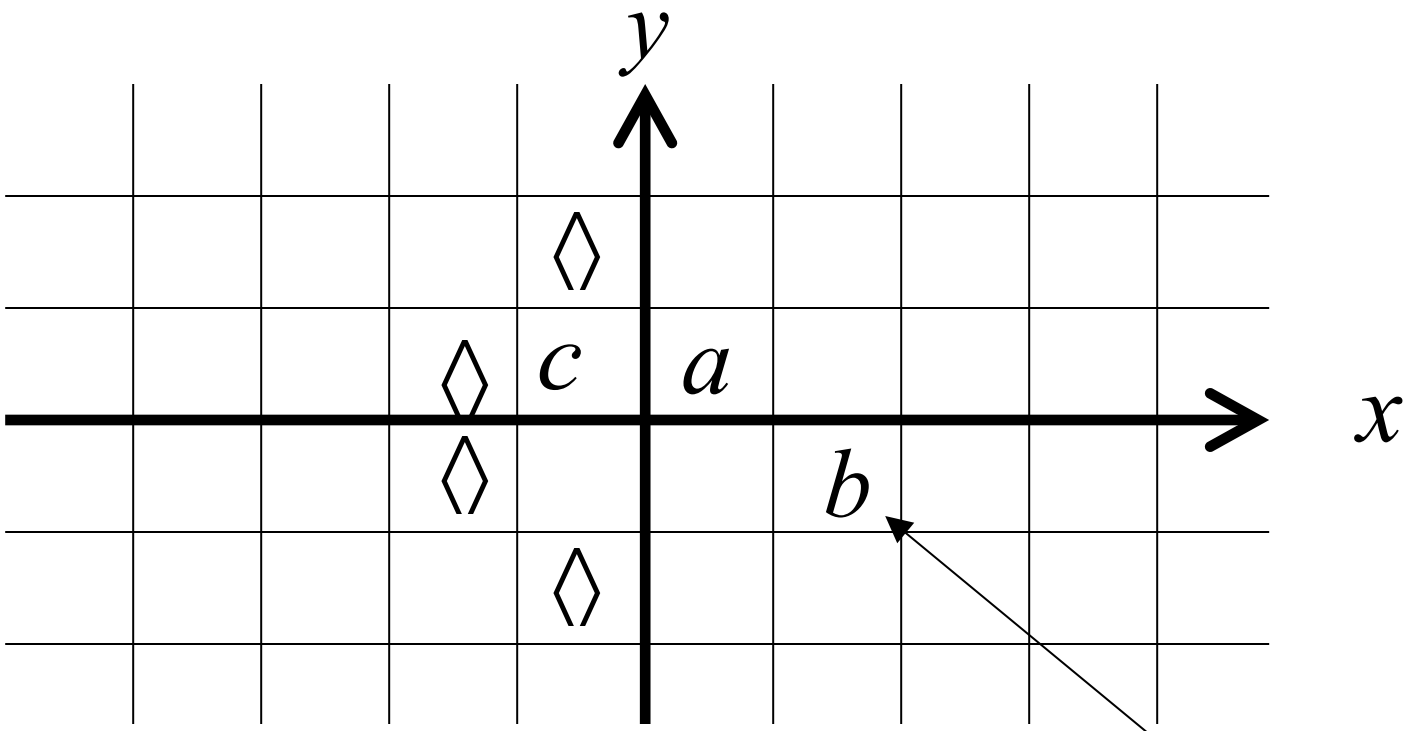
Position: +2, -1

2. Standard Turing machines simulate Multidimensional machines

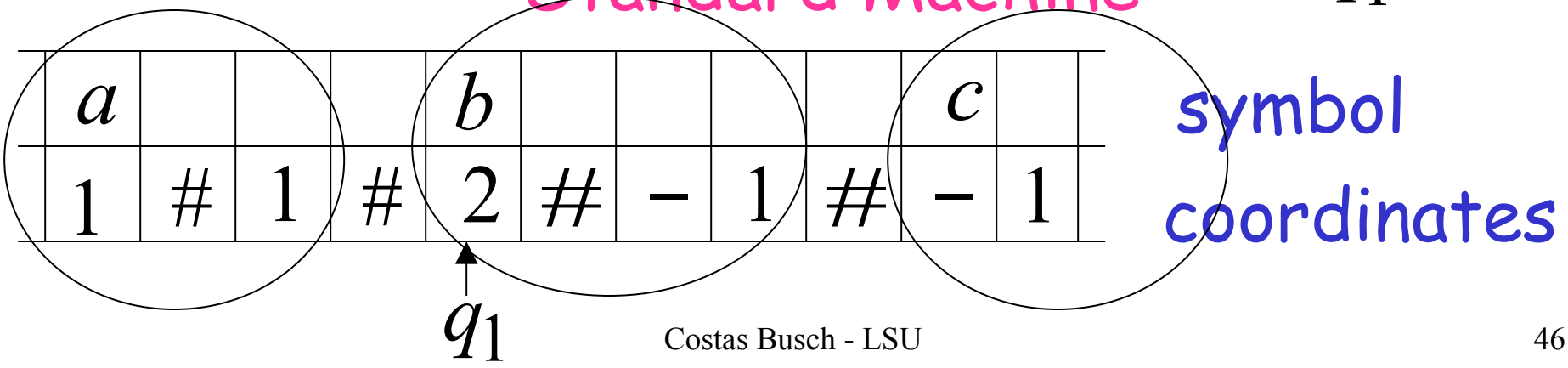
Standard machine:

- Use a two track tape
- Store symbols in track 1
- Store coordinates in track 2

2-dimensional machine



Standard Machine



Standard machine:

Repeat for each transition followed
in the 2-dimensional machine:

1. Update current symbol
2. Compute coordinates of next position
3. Go to new position

END OF PROOF

Oblivious Turing machines

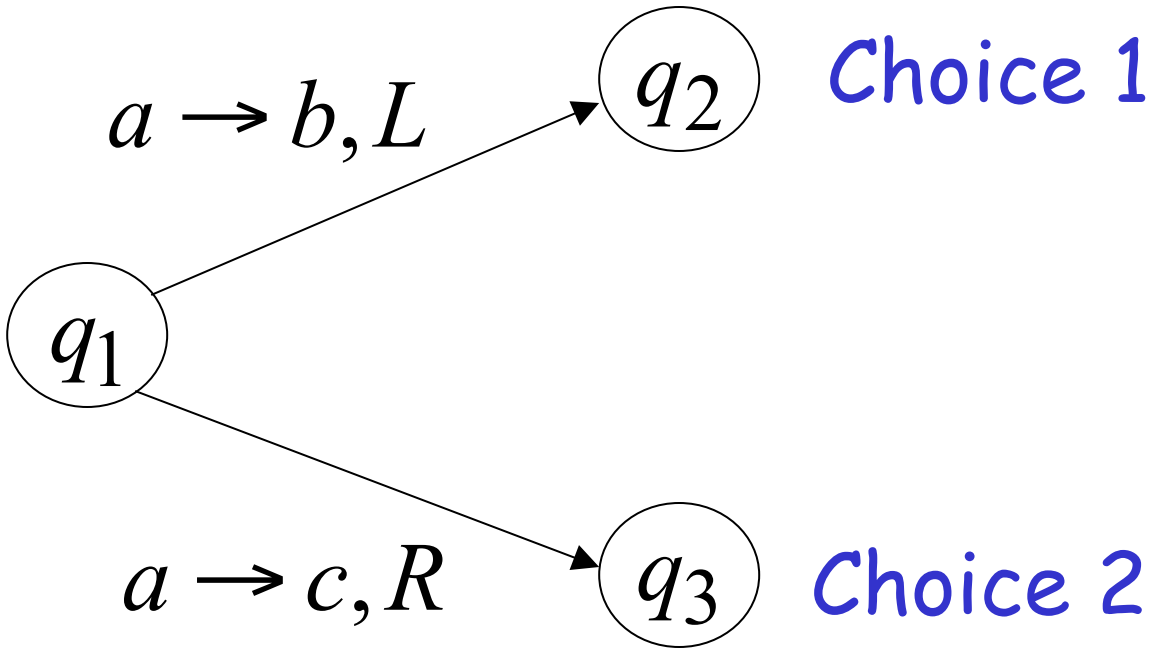
Definition (Oblivious Turing Machine)

A multi-tape Turing machine M is **oblivious** if the movement of its heads depends only on the input length n and the time step t, but not on the input contents.

More precisely:

- For each tape i and time step $t \leqslant T(n)$, there is a fixed position $p_i(t, n)$ where head i must be located.
- The actual computation (state transitions, symbols written) can still depend on the input contents.
- Every T-time multi-tape Turing machine can be made oblivious in time $O(T \log T)$.

Nondeterministic Turing Machines



Allows Non Deterministic Choices

Input string w is accepted if there is a computation:

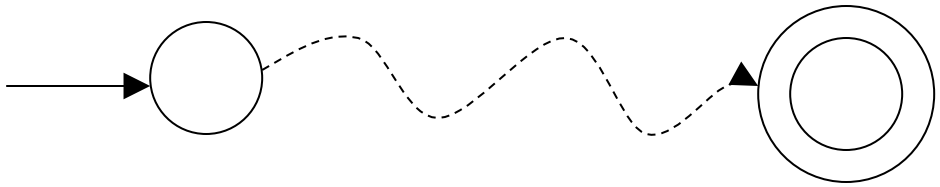
$$q_0 w \overset{*}{\succ} x\, q_f\, y$$

Initial configuration

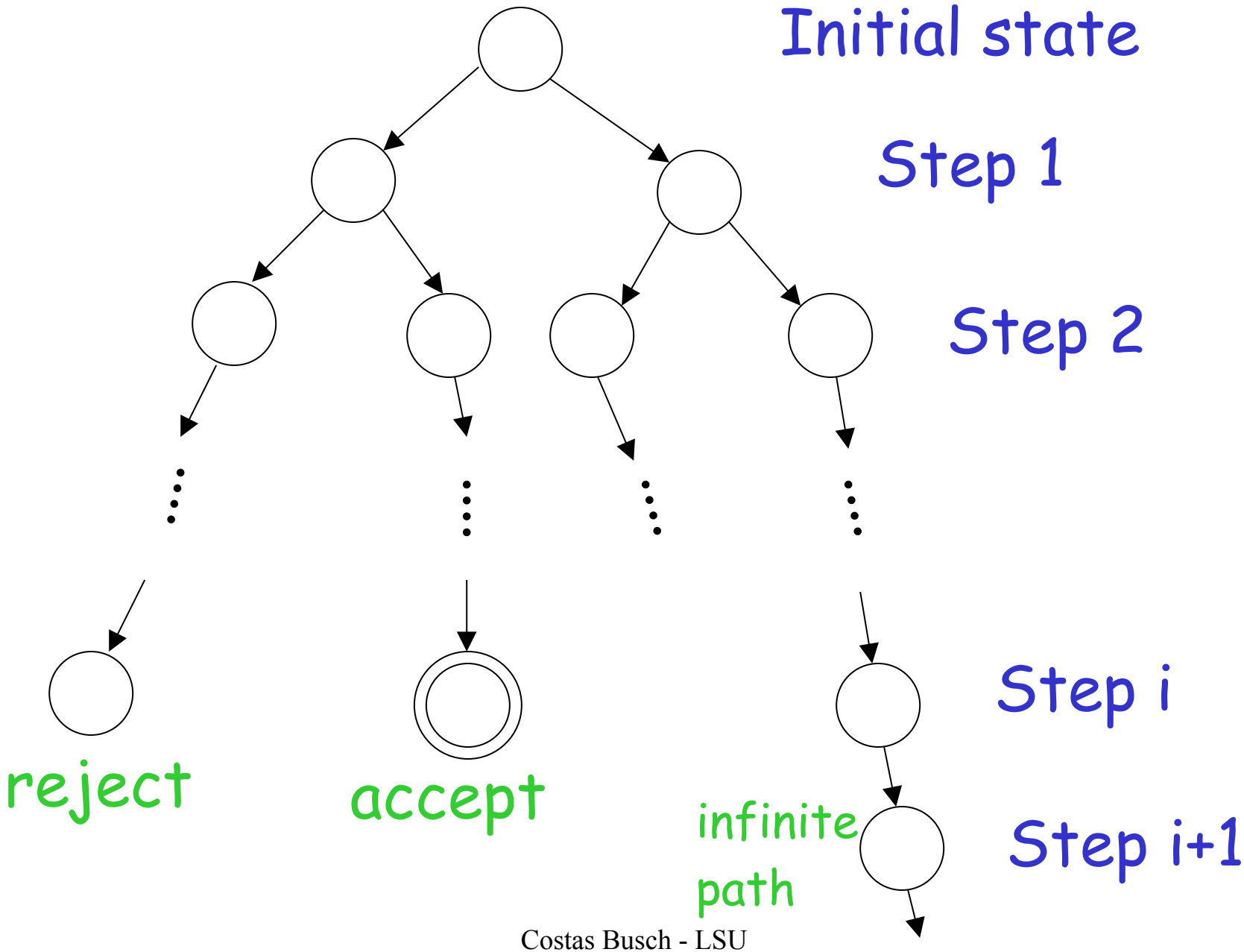
Final Configuration

Any accept state

There is a computation:



All possible computation paths



The Deterministic Turing machine simulates all possible computation paths:

- simultaneously
- step-by-step
- with breadth-first search

depth-first may result getting stuck at exploring an infinite path before discovering the accepting path

Universal Turing Machine

There is a Turing machine U which given the encoding of a Turing machine M and an input x can simulate the execution of M on x and (a) Accept if M accepts x, and (b) Reject if M rejects x.
U is called the universal Turing machine.
Since U is a fixed machine, its tape alphabet is fixed. However, it needs to be able to simulate TMs with an arbitrary tape alphabet. This is achieved by encoding the TMs using a fixed alphabet.

Encoding Turing Machines

Consider an arbitrary Turing machine $M = (Q, \Sigma, \Gamma, \vdash, \sqcup, \delta, s, t, r)$.

- We will encode M using the alphabet $\{0, 1, [,], \bullet, |\}$.
- Encode each $a \in \Gamma$ as a binary string $\text{enc}(a)$ of length $\log |\Gamma|$; we will assume that $\vdash = 0^{\log |\Gamma|}$ and $\sqcup = 0^{\log |\Gamma| - 1}1$
- A string $x = a_1 a_2 \cdots a_n \in \Gamma^*$ will be encoded as $[\text{enc}(a_1) \bullet \text{enc}(a_2) \bullet \cdots \bullet \text{enc}(a_n)]$
- Each state $q \in Q$ is encoded as a binary string $\text{enc}(q)$ of length $\log |Q|$; we will assume that $s = 0^{\log |Q|}$, $t = 0^{\log |Q| - 1}1$ and $r = 0^{\log |Q| - 2}10$
- Directions L and R will be encoded as 0 and 1, respectively.

Turing Machine Codes

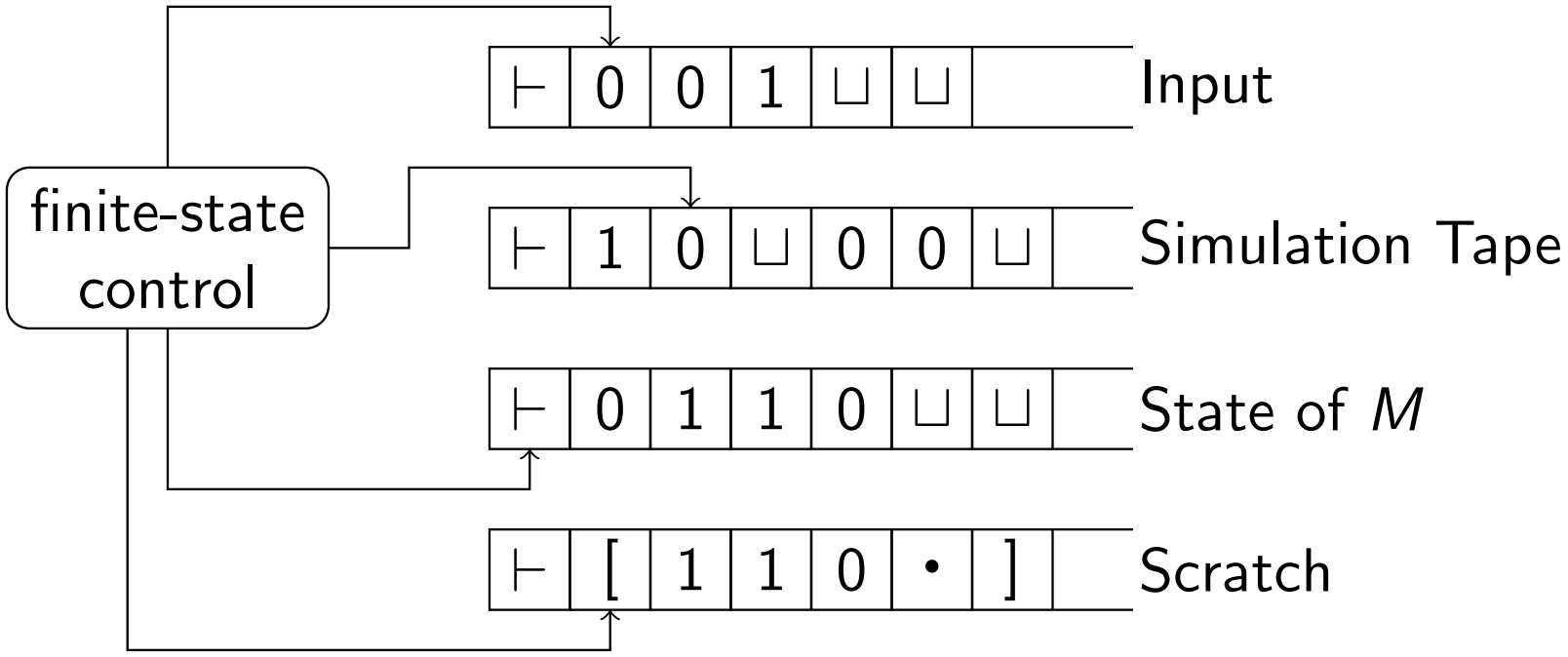
Continued

- Each transition $\delta(p, a) = (q, b, d)$ is encoded as $[\text{enc}(p) \bullet \text{enc}(a) | \text{enc}(q) \bullet \text{enc}(b) \bullet \text{enc}(d)]$
- The code for the machine itself is

$$[\underbrace{\quad}_{\text{trans 1}}\underbrace{\quad}_{\text{trans 2}} \cdots\cdots \underbrace{\quad}_{\text{last trans}}]$$

- We will denote the encoding of machine M and input x as $\langle M, x \rangle$

Universal Turing Machine



Schematic picture of Universal TM

U will store the configuration of M by storing, the state of M on the state tape, and the tape of M on the simulation tape.

UTM algorithm

1. Check to see if the code for M is a valid TM code; if not reject the input. For example, for our code, it involves checking to see if the string as exactly one "|" between [and], etc.
2. If code is ok, then copy x onto tape 2
3. Write $0 \cdots 0$, the start state of M, on the third tape, and scan the "first cell" of tape 2.
4. To simulate a move of M, search for a transition $[\text{enc}(p) \bullet \text{enc}(a) | \text{enc}(q) \bullet \text{enc}(b) \bullet \text{enc}(d)]$ on tape 1, where $\text{enc}(p)$ is on tape 3 (current state) and $\text{enc}(a)$ is read from tape 2 from the current "cell", i.e., between two successive $\bullet$ symbols from the current head position. Then, write $\text{enc}(q)$ on tape 3 (after erasing its current contents), write $\text{enc}(b)$ on tape 2 instead of $\text{enc}(a)$, and finally move the head on tape 2 to the appropriate "cell".
5. If state on tape 3 is $0 \cdots 01$ then accept; if state is $0 \cdots 010$ then reject.